

Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 2)

Topics: Spinlocks, Interrupt-safe Locking, SMP and Best Practices

Spinlocks

Spinlocks protect shared kernel data. If the lock is unavailable, the CPU spins until the lock becomes free. They are intended for very short critical sections where sleeping is not allowed.

Use Cases

- Interrupt handlers
- Bottom halves
- SMP synchronization
- Fast shared data protection

Avoid Spinlocks For

Blocking operations, long-running code, disk I/O, network I/O or any code that may sleep.

API	Purpose
<code>spin_lock()</code>	Acquire
<code>spin_unlock()</code>	Release
<code>spin_lock_irq()</code>	Disable interrupts
<code>spin_lock_irqsave()</code>	Save IRQ state
<code>spin_unlock_irqrestore()</code>	Restore IRQ state
<code>spin_lock_bh()</code>	Disable bottom halves

Interrupt-safe Locking

Use `spin_lock_irqsave()` when the same lock may be taken in both process and interrupt contexts. It prevents deadlocks by disabling local interrupts while the lock is held.

SMP Example

Multiple CPUs updating the same counter simultaneously can lose updates. A spinlock serializes access to guarantee correctness.

Common Mistakes

- Sleeping while holding a spinlock.
- Forgetting to unlock.
- Holding locks for too long.
- Using spinlocks where a mutex is appropriate.

Debugging

Use `dmesg`, `lockdep`, `ftrace`, `CONFIG_DEBUG_SPINLOCK` and `CONFIG_PROVE_LOCKING` to identify locking issues.

Interview Questions

1. What is a spinlock?
2. Why can't spinlocks sleep?
3. Spinlock vs mutex?
4. Explain `spin_lock_irqsave()`.
5. What is `spin_lock_bh()`?
6. How do you debug spinlock deadlocks?

Key Takeaways

Spinlocks are the preferred synchronization primitive for short, non-sleeping critical sections in kernel and interrupt contexts. Correct API selection is critical for safe concurrent execution.